

Updating the World Is Not Rebinding the Target: Temporal Referent Integrity in Tool-Using Agents

Anonymous submission

Abstract

Tool-using language agents repeatedly refresh external state while carrying out a user’s request. A refreshed observation, however, does not by itself authorize the agent to change the entity denoted by an earlier reference. We formalize this post-binding authorization problem as *temporal referent integrity* (TRI): belief update and referent transition are separate controller decisions. On a frozen 160-task minimal-pair diagnostic, a Generic Structured Ledger that retains the instruction, selected ID, snapshot, selector, and action preconditions reaches 64.4% and 71.9% with Qwen3.5-122B and GLM-5.1. Explicit pre-refresh Compile-then-act reaches 95.0% and 96.2%, recovering most of the observed gap and supporting pre-refresh commitment compilation as the leading mechanism. A typed Lifecycle representation with deterministic mutation gating reaches 98.1% and 100.0%; the gate contributes only 1.2–1.9 points over a matched free actor, while making validity and fallback enforceable. Across two 40-trajectory SQLite replays, Generic state produces 13 and 8 wrong-entity writes; Lifecycle-Gated reaches 40/40 final states for both models with none. On 50 rewrites by an independent volunteer, minimal CTA reaches 90/98% and Gated 90/94%. Three blind annotators on 100 original/natural-rewrite items obtain Fleiss’ $\kappa = .708$ and 86% majority-gold agreement. Majority-gold alignment falls to 55% on action-invalid reject outcomes, showing that fallback is a normative execution policy rather than settled referential semantics. TRI therefore contributes a controlled diagnosis and executable authorization contract, not a claim of universal autonomous-agent safety.

Introduction

Consider two requests issued before the same mailbox refresh:

- (1) “Choose the highest-priority unread email now. Refresh the mailbox, then reply to it.”
- (2) “Refresh the mailbox first. Then choose the highest-priority unread email and reply.”

Suppose email A is highest priority before refresh and email B afterward. The environment transition is identical, but the correct target differs. Request (1) establishes A as

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

an identity commitment; request (2) deliberately leaves a description to be evaluated in the new state. Acting on B in (1) is not stale-state correction. It is a wrong-entity action.

This distinction is easy to lose in LLM agent controllers. A controller may preserve the entire transcript, append the latest observation, and ask a downstream model to act. All facts remain available, yet the model can silently apply the old selector to the new state. Conversely, always retaining an old ID breaks genuinely dynamic references. Reliable tool use therefore needs *selective invariance*: world facts should update while some target denotations remain fixed, others are reevaluated, and invalid commitments follow an explicit fallback policy.

We call this property *temporal referent integrity* (TRI). TRI is not ordinary entity memory. Remembering an ID does not say whether that ID is a committed referent or merely the previous output of a dynamic selector. Nor does existence imply action validity: an email may remain present but cease to be replyable. Finally, the selector that established identity is not necessarily a persistent validity predicate. A branch chosen because it was default can stop being default while remaining the correct branch on which to run checks.

Figure 1 summarizes the problem and our controller. A Generic Structured Ledger records identity, facts, selector, and action preconditions, while leaving the actor to infer how those fields interact across refresh. Minimal Compile-then-act (CTA) first records binding time and identity. Its typed Lifecycle-Gated extension adds validity and fallback state; a deterministic gate enforces preserved identities while an actor resolves dynamic branches.

Our contributions are threefold:

- We formalize TRI as a post-binding authorization problem separating reference mode, bound identity, selector provenance, action-relative validity, and guarded fallback. Latest-state observations can be informationally insufficient; full history may contain the answer while still leaving the commitment computationally implicit and unenforced.
- We introduce frozen clustered diagnostics for language-template clusters, unseen schemas, and guarded conditional commitments. We pre-specify cluster-resampled statistics and audit API failures, missing tasks, retries, and stage-wise errors.

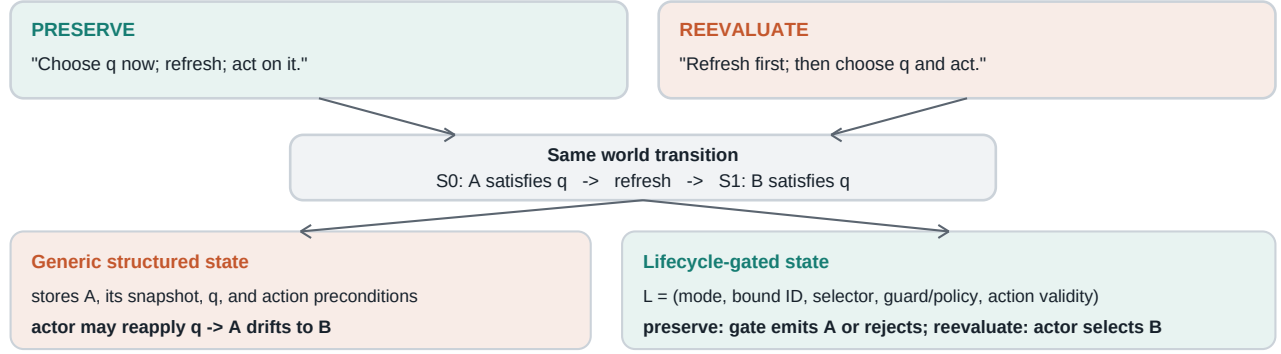


Figure 1: The same state transition requires different targets depending on discourse. Generic structured state preserves facts but leaves the interaction between identity and selector implicit. Lifecycle state makes the permitted denotation update executable.

- Across two models, independent human rewrites, and SQLite mutations, component evidence most strongly supports pre-refresh commitment compilation. Minimal CTA has the highest or tied scalar point estimate; typed lifecycle state exposes a richer authorization contract, while negative stress tests localize its compilation and compositional costs.

Related Work

Reference and entity binding. Discourse Representation Theory and centering distinguish discourse referents from changing descriptions and salience (Kamp and Reyle 1993; Grosz, Joshi, and Weinstein 1995). Neural coreference models and Quoref recover textual identity links (Lee et al. 2017; Joshi et al. 2020; Dasigi et al. 2019). Entity Binding Failures studies wrong initial tool arguments, provenance, clarification, and confidence gating (Babu and Indukuri 2026). These works ask whether an entity was grounded correctly from the available context. Entity Binding Failures therefore asks whether $bind(r, S_0)$ is correct; TRI conditions on a correct binding having occurred and asks whether observing S_1 licenses a denotation transition, a question that arises only after the binding event.

Entity tracking and agent memory. Recent entity-tracking work tests whether language models preserve entity identity while facts change across state transitions (Tang et al. 2026). Memory systems retain summaries, experiences, skills, or retrieved traces, including Generative Agents, Reflexion, Self-Refine, Voyager, MemGPT, MemoryBank, and AutoGen (Park et al. 2023; Shinn et al. 2023; Madaan et al. 2023; Wang et al. 2023; Packer et al. 2023; Zhong et al. 2024; Wu et al. 2023); surveys and cognitive-architecture work organize these choices (Zhang et al. 2024; Summers et al. 2024). ReAct interleaves reasoning and acting (Yao et al. 2023). TRI isolates a missing relationship in such state: whether a selector remains executable after refresh or has become provenance for a committed identity.

Tool-use and interactive benchmarks. Toolformer, API-Bank, ToolLLM, and Gorilla study API acquisition and tool

calling (Schick et al. 2023; Li et al. 2023; Qin et al. 2024; Patil et al. 2023). AgentBench, WebArena, VisualWebArena, WebShop, GAIA, SWE-bench, and MINT evaluate agents in web, coding, multimodal, and feedback-rich environments (Liu et al. 2024; Zhou et al. 2024; Koh et al. 2024; Yao et al. 2022; Mialon et al. 2024; Jimenez et al. 2024; Wang et al. 2024). ToolSandbox, AppWorld, and τ -bench add stateful applications, policies, and user interaction (Lu et al. 2025; Trivedi et al. 2024; Yao et al. 2024); ToolEmu studies agent risk (Ruan et al. 2024). Dynamic-environment suites such as ClawArena and EvoArena focus on evolving information and memory robustness (Ji et al. 2026; Xu et al. 2026). Temporal Blindness and TicToc instead study whether elapsed time should trigger a tool refresh (Cheng et al. 2026); TRI conditions on an update having occurred and asks whether that update authorizes changing an already established action referent. TRI complements these benchmarks with controlled temporal-target minimal pairs and a focused SQLite mutation test rather than claiming coverage of their broader environments.

Executable constraints and diagnostic evaluation. LMQL, guided generation, and DSPy structure model calls or compile declarative programs (Beurer-Kellner, Fischer, and Vechev 2023; Willard and Louf 2023; Khattab et al. 2024). LedgerAgent combines typed state with policy checks, while Bounded Autonomy intercepts wrong-entity mutation through typed action contracts at the consumer-side execution boundary (Uddin et al. 2026; Sohail and Haider 2026). These schema-derived checks ask whether a proposed action is permitted for its proposed entity. TRI instead asks the earlier discourse-derived question: whether selection in S_0 established an identity commitment before refresh, or left a description to be evaluated in S_1 . Execution-time validity remains necessary but does not authorize substituting a different valid entity. We do not claim the first ledger, typed contract, or gate. Finally, CheckList and contrast sets motivate our clustered minimal-pair evaluation (Ribeiro et al. 2020; Gardner et al. 2020).

Problem Formulation: Temporal Referent Integrity

Let u be an instruction, r its action-target reference, S_0 the state before refresh, S_1 the refreshed state, and a the requested action. We factor the executable contract into a referential lifecycle R and an action policy P :

$$\begin{aligned} L(r, a) &= (R(r), P(a)), \\ R &= (m, e, q, g), \quad P = (V_a, \pi), \end{aligned} \quad (1)$$

where m is a reference mode, e is an identity bound in S_0 , q is a selector, g is an optional persistence guard, π is a fallback policy, and $V_a(e, S)$ is validity under the requested action schema. The main TRI-v3 setting uses $m \in \{\text{PRESERVE}, \text{REEVALUATE}\}$ and an explicit reject policy for an invalid preserved target. TRI-v4 adds conditional guards.

For the scalar policies evaluated here, the execution branches are

$$\text{PRESERVE}, V_a(e, S_1) \Rightarrow e, \quad (2)$$

$$\text{PRESERVE}, \neg V_a(e, S_1), \pi = \text{REJECT} \Rightarrow \perp, \quad (3)$$

$$\text{REEVALUATE} \Rightarrow q(S_1), \quad (4)$$

$$\text{CONDITIONAL}, g(e, q, S_1) \Rightarrow e, \quad (5)$$

$$\text{CONDITIONAL}, \neg g(e, q, S_1) \Rightarrow q(S_1). \quad (6)$$

An invalid preserved entity remains the discourse referent e under R even when P makes the authorized action output $\perp$. Thus rejection is an execution-policy decision, not a claim that the reference denotes nothing. The human study below empirically motivates this factorization: referential transitions receive stronger agreement than invalid-target handling. TRI-v4 distinguishes $g_{\text{act}} = V_a(e, S_1)$ from $g_{\text{sel}} = [q(S_1) = e]$. This matters when a refresh changes the selector winner but the original entity remains action-valid: g_{act} preserves it, whereas g_{sel} rebinds.

Three insufficiency pairs motivate this factorization. First, two tasks can share S_1 , q , and a but differ only in whether the discourse selected before or after refresh. Latest-state control cannot recover the missing commitment. Second, two tasks can store the same old ID, but in one it is committed and in the other it is merely a stale dynamic-selector result; an ID without a role annotation cannot choose correctly for both. Third, two tasks can share mode and ID while requesting actions with different preconditions. The target may be valid for viewing but not for canceling, shipping, or approving. Validity is therefore $V_a(e, S)$, not mere existence.

These are three distinct claims. An observation may be *informationally insufficient* because a binding event was discarded. A full transcript or ledger may be *informationally sufficient* but leave the role of an ID *computationally implicit* in prose. Even a model that infers the intended role correctly does not provide *enforcement*: a later actor can still violate that decision unless the mutation boundary checks it. TRI targets the latter two failure modes without claiming that full history is unable to represent the answer.

The selector q and validity V_a have distinct roles. In a preserved reference, q was used to establish identity in S_0 ; reapplying it after refresh is precisely the error under study.

Only the action schema determines whether the committed entity can still receive a . Conditional language can explicitly reactivate q , but that permission must be represented rather than inferred afresh by every downstream actor.

The contract makes the authorization boundary explicit. Let $\mathcal{A}(L, S_1)$ be the set of targets licensed after refresh:

$$\mathcal{A}(L, S_1) = \begin{aligned} &\{e\}, && m = \text{PRESERVE}, \\ &\{q(S_1)\}, && V_a(e, S_1) \quad, \\ &\{e\}, && m = \text{REEVALUATE}, \\ &\{q(S_1)\}, && m = \text{CONDITIONAL}, \\ &\{q(S_1)\}, && g(e, q, S_1) \quad, \\ &\{q(S_1)\}, && m = \text{CONDITIONAL}, \\ &\{q(S_1)\}, && \neg g(e, q, S_1) \quad, \\ &\{q(S_1)\}, && m = \text{PRESERVE}, \\ &\{q(S_1)\}, && \neg V_a(e, S_1), \pi = \text{REEVALUATE?} \\ &\{\perp\}, && m = \text{PRESERVE} \wedge \neg V_a(e, S_1), \\ &&& \pi = \text{REJECT}. \end{aligned} \quad (7)$$

TRI compliance requires a proposed target x to belong to $\mathcal{A}(L, S_1)$. Mutation authorization is a second check: a non-rejection proposal may execute only if $V_a(x, S_1)$. If it fails, the mutation boundary blocks it; action validity alone never authorizes substituting a different valid target. Whether the blocked outcome is surfaced as rejection, clarification, or another explicit fallback belongs to P , not to referential semantics. This two-layer condition applies only after the controller has compiled the correct mode, identity, guard, and fallback from the instruction; it does not assume an infallible LLM compiler.

Proposition (state-overwrite indistinguishability). Consider a controller whose post-refresh state is exactly (u, S_1) , where u is the original instruction and S_1 is the refreshed observation. There exist two histories with the same anchored instruction u and the same S_1 , but different initial states S_0, S'_0 such that $\text{bind}(r, S_0) = e$, $\text{bind}(r, S'_0) = e'$, and $e \neq e'$. If both histories refresh to S_1 , the controller receives the same post-refresh input while the correct targets differ. Therefore any deterministic state-overwrite controller is wrong on at least one history. This matches the tested baseline: the temporal instruction remains in the input, but the earlier binding result is not recoverable from the common latest state. The construction is a diagnostic lower bound, not a claim that every full-history or learned-memory controller fails.

Proposition (authorization and mutation validity). Assume (i) L correctly encodes the user's authorized mode, identity, selector, guard, and fallback; (ii) entity IDs remain stable across the evaluated transition; (iii) selector and guard evaluation are correct and V_a completely implements the action schema; and (iv) the executor follows the branches defining $\mathcal{A}(L, S_1)$. Then every proposed non-rejection target belongs to $\mathcal{A}(L, S_1)$. If the mutation tool executes only proposals satisfying V_a , every executed mutation is both referentially authorized and action-valid; an authorized but invalid proposal is blocked rather than silently replaced. The first claim follows by cases on m , g , and π , and the second by the mutation precondition. The guarantee ends at the compiled contract: a wrong but action-valid bound ID can still pass, and identity migration or an incomplete action schema violates the assumptions.

Safety-sensitive outcomes. Exact target accuracy treats every mismatch equally, but the operational consequences

differ. A *wrong-target attempt* names an entity other than the gold target; if that entity is present and action-valid, it becomes a *wrong-entity write*. An *invalid-target attempt* names an entity that violates action preconditions and should be blocked by the tool. An *unnecessary rejection* returns $\perp$ when a valid target exists. The first two can modify the wrong external record; the last is conservative loss of utility. We therefore report resolution accuracy, final database state, wrong writes, invalid attempts, and unnecessary rejections separately. TRI does not define safety as refusal: a controller that always rejects would avoid wrong writes but fail every actionable task.

Method: Pre-Refresh Commitment Compilation

The controller family first compiles (u, S_0) before refresh. Minimal CTA emits binding time, selector, and a bound ID. The typed Lifecycle extension additionally emits

```
reference_mode; bound_target_id;
selector; invalidity_policy.
```

Both compilers use discourse order: “identify, refresh, act on it” preserves identity even without words such as “same,” while “refresh before deciding” reevaluates. It resolves a concrete bound ID only for preserved references.

The executor then refreshes the environment. If the compiled mode is PRESERVE, a deterministic gate checks the bound entity against action preconditions. The gate emits the bound ID when valid, returns $\perp$ under a reject policy, or invokes selector resolution only when the compiled fallback authorizes it. If the mode is REEVALUATE, the LLM actor selects from S_1 . Thus a preserved commitment is a controller constraint, not advice that a second model may reinterpret. The preserve branch also needs one model call rather than two.

TRI-v4 generalizes the record with `guard_type` and `fallback_policy`. The action-validity guard can be enforced deterministically. A selector-match guard is evaluated by the actor because the current prototype compiles natural language selectors but does not execute them as a general symbolic query language. This design choice exposes selector grounding as a measurable stage rather than using gold selector fields.

The complete controller procedure and model-facing interfaces are provided in the supplement. Gold targets and benchmark selector fields are never provided to the compiler or actor.

Controller Information and Fairness

Table 1 separates the information boundary, task decomposition, and enforcement mechanism. Except for the unchanged historical predecessor, each compiler sees the instruction, S_0 , action schema, and the fact that refresh precedes action. Generic also retains all task-relevant evidence for its post-refresh actor; its weakness is therefore behavioral accessibility, not information loss. Untyped and Lifecycle-free are the closest two-call representation comparison. Historical Compile-then-act is a strong predecessor but omits action validity from its actor interface.

Controller	Pre-refresh record	Calls	Gate
Generic ledger	ID, snapshot, selector, action, pre-conditions	2	no
Untyped plan	one free-form plan	2	no
Historical CTA	binding time, ID, selector, reason	2	no
Lifecycle free	mode, ID, selector, policy	2	no
Lifecycle gated	same typed record	1 or 2	yes

Table 1: Controller audit. Calls are per task: the gated Preserve branch skips the actor. Prompts, exact actor inputs, and the executor-level oracle sufficiency check are supplementary.

When task-generator fields are supplied directly, a deterministic full-lifecycle executor reaches 100% on the 246-task TRI-v2 suite, whereas reduced records reach 41.9–74.0%. This is only an executor-level representation sufficiency check under generated semantics, not evidence that lifecycle fields are minimal or that an LLM can infer them. Full results are reported in the supplement rather than used as method-performance evidence.

Experimental Setup

Controlled task construction. Each domain specification defines an entity table, a natural-language selector, its ranking or Boolean criterion, an action, and explicit action preconditions. The generator first evaluates the selector on S_0 to obtain e_0 , applies a controlled transition, and evaluates it on S_1 to obtain e_1 . A language template determines whether the action target is e_0 , e_1 , $\perp$, or a conditionally guarded choice. Paired tasks share the domain, both states, selector, action, and schema; only discourse order and anaphoric language change. This prevents domain difficulty or target frequency from explaining the preserve/reevaluate contrast.

The five transition types serve different diagnostic roles. *Flip* changes which valid entity wins the selector. *Stable* leaves the winner unchanged and detects controllers that overreact to any refresh. *Remove* deletes the pre-refresh entity. *Invalidate* keeps its ID present while changing an action-precondition field, separating existence from validity. *Name collision* gives another entity the same display label while changing the selector winner, testing whether controllers preserve stable identity rather than surface aliases. Across anchored and dynamic language, these updates create matched cases where naive “always old” and “always new” policies have complementary failures.

Blind human construct validation. One independent volunteer naturally rewrote 50 sampled scalar instructions. Three other volunteers, uninvolved in TRI design, independently labeled 100 randomized original/rewrite items using opaque IDs without condition metadata or gold answers. They selected an entity, REJECT, or CLARIFY; confidence was optional. We report majority–gold alignment, unanimous and pairwise agreement, Fleiss’ κ , nominal Krippendorff’s α , and a consensus-only sensitivity analysis. Conditional, collection, and nested cases were not included in this human study.

Controller	Qwen3.5	GLM-5.1
Generic free actor	64.4	71.9
Generic + reference mode	75.0	75.0
Generic + validity gate	65.0	73.1
Untyped pre-refresh plan	81.2	70.6
Exact compile then act	95.0	96.2
Lifecycle free actor	96.9	98.1
Lifecycle + gate	98.1	100.0

Table 2: Exact target accuracy (%) on the same 160-task v3 language set. The post-freeze Generic+reference-mode ablation adds only the mode field; Lifecycle-free is statistically tied with historical Compile-then-act, and the gate adds only 1.2–1.9 points.

Stage and write-consequence reports use a frozen mechanism-oriented error taxonomy, detailed in the supplement, that separates temporal rebinding, premature binding, invalid processing, unnecessary rejection, and selector-grounding errors.

Strong baselines and component audit. Table 2 reports the frozen v3 primary inventory. Generic validity gating, an untyped pre-refresh plan, Lifecycle-free execution, and exact historical Compile-then-act were post-primary audits; they do not replace the pre-specified Generic-versus-Lifecycle-Gated estimand. They do, however, determine how narrowly that comparison can be interpreted.

TRI-v3 language clusters. The primary test contains 160 tasks formed from 20 frozen language-template clusters, four reference styles, and eight domains per template. Within each style, flip, stable, remove, invalidate, and name-collision updates are balanced. The pre-specified statistic resamples whole template clusters with 10,000 bootstrap samples; task-level McNemar is secondary. Before any model call, we froze the data, runner, hashes, inference settings, and rule that prompts would not change after output inspection.

Unseen schemas and guarded policies. An 80-task transfer set crosses the same 20 template clusters with four unseen domains: projects, expenses, inventory, and cloud deployments. These introduce new IDs, fields, selectors, actions, and action preconditions. TRI-v4 separately freezes 40 conditional tasks, ten template clusters, and the two guards g_{act} and g_{sel} . Every guard-update-domain combination occurs once. TRI-v4 is a secondary extension, not part of the pre-specified v3 primary comparison.

Direct-neighbor baseline. Generic Structured Ledger uses two calls. Its compiler stores the initial selected ID, complete entity snapshot, selector, requested action, and action preconditions, but omits reference mode and invalidity policy. Its actor still receives the original instruction, ledger, S_1 , and action schema. It is therefore not an information-deletion baseline: the comparison asks whether structured facts suffice without discourse commitment semantics.

The historical Compile-then-act prompt is rerun unchanged on the v3 language set (Table 2). Its cluster interval overlaps lifecycle-free execution for both models, consistent

with pre-refresh compilation, rather than tuple serialization or deterministic gating alone, being the leading explanation. Because it emits binding time and a bound ID, it also serves as the simple mode-classifier-plus-ID-lock analogue for scalar cases. The typed contract contributes an auditable interface for validity and fallback; the gate is a small enforcement layer, not the principal accuracy mechanism. A matched free-form pre-refresh planner reaches only 81.2% for Qwen and 70.6% for GLM, so moving computation before refresh is not alone sufficient; nevertheless, the strong historical predecessor prevents us from claiming that the full tuple is empirically necessary for scalar tasks.

Model-facing SQLite trajectories. We freeze a 40-task subset covering all 20 templates, eight domains, both modes, and all update types. The runtime queries an in-memory SQLite database, gives returned rows to the compiler, replaces the database state during refresh, obtains a target from the gate or actor, and executes UPDATE through a mutation tool that enforces action preconditions. We inspect final database state, wrong writes, invalid attempts, and collateral modifications. Query and required refresh order are controller-orchestrated; this is a focused execution test, not a fully autonomous external benchmark.

Models, inference, and auditing. We use Qwen3.5-122B-A10B and GLM-5.1 through the same OpenAI-compatible API at temperature zero, with thinking disabled and a 1,200-token output cap. Balanced smoke tests gate each expansion. Final reports verify expected task IDs, duplicates, status, request attempts, and retries. All reported v3/v4 full files are complete and contain zero API errors and zero retries. API failures would be reported separately rather than interpreted as model behavior.

Statistical estimands. The primary estimand is Lifecycle-Gated minus Generic Structured Ledger accuracy on the Qwen language-cluster set. We resample its 20 complete template clusters rather than treating 160 templated rows as independent. The same cluster bootstrap is reported for replication and transfer sets; each interval uses 10,000 samples and a fixed recorded seed. Template-macro and task accuracy coincide in the balanced sets, but both are retained in machine-readable reports. Exact paired McNemar tests use task-level discordance and are secondary. Wilson intervals describe individual proportions. Slices by binding, explicitness, update, and domain are mechanism analyses rather than additional confirmatory hypotheses; we do not use them to select the primary claim.

Reproducible run discipline. Frozen data, protocols, runners, and smoke subsets have recorded SHA-256 hashes. Reports retain raw outputs, attempts, retries, stage errors, and SQLite traces; interrupted rows are auditable by ID. The anonymous artifact maps every main-table value to a raw run and report script, includes the human-analysis code and de-identified responses, and excludes API keys and private coordination files. Post-primary audits are labeled as such.

Results

Primary and Transfer Results

Structured facts are not a commitment. Table 3 and Figure 2 show the same mechanism across models. On the primary language-cluster test, Generic Structured Ledger reaches 64.4% for Qwen and 71.9% for GLM, despite carrying identity, facts, selector, preconditions, and the original instruction. Lifecycle-Gated reaches 98.1% and 100.0%. Paired discordance is 3 generic-only versus 57 lifecycle-only for Qwen ($p = 6.25 \times 10^{-14}$), and 0 versus 45 for GLM ($p = 5.68 \times 10^{-14}$). Qwen improves from 33.8% to 100.0% on anchored references while remaining comparable on dynamic references (95.0% versus 96.2%). GLM improves from 56.2% to 100.0% anchored and from 87.5% to 100.0% dynamic. The gain is therefore not explained by always preserving old IDs.

Because the Lifecycle compiler receives explicit reference-mode and invalidity semantics, this comparison does not by itself identify the causal contribution of serialization, prompt supervision, and the deterministic gate separately. The historical compile-then-act results in Table 2 provide a strong neighboring comparison and motivate the narrower interpretation: explicit pre-refresh compilation is the main performance mechanism, while lifecycle state makes its authorization and validity decisions auditable and enforceable.

The post-primary component audit sharpens this conclusion. A validity-only gate changes Generic accuracy by only +0.6 points for Qwen and +1.2 for GLM. Lifecycle state with a free actor reaches 96.9% and 98.1%; deterministic enforcement adds only +1.2 and +1.9 points. The new Generic+reference-mode ablation reaches 75.0% for both models, with paired cluster differences of +10.6 [+5.0, +16.9] and +3.1 [-2.5, +9.4] points. Thus explicit mode classification is useful but model-dependent and insufficient to explain CTA’s full gain. The call-matched untyped plan reaches 81.2% and 70.6%, 15.6 and 27.5 points below Lifecycle-free with cluster intervals excluding zero, showing that pre-refresh timing alone is insufficient. Exact historical CTA reaches 95.0% and 96.2%, tied with Lifecycle-free; the audit therefore supports commitment compilation as the main mechanism, with typed lifecycle state adding auditable validity and fallback rather than explaining the full accuracy gap.

Because the aggregate includes a normative invalid-target policy, we also separate its 128 actionable core items: Generic/Historical-CTA/Gated reach 74.2/98.4/97.7% for Qwen and 72.7/99.2/100.0% for GLM. The complete 32-item policy slice is supplementary; this post-primary sensitivity does not replace pre-specified total accuracy.

The strongest generic failures occur when the bound entity remains valid but no longer wins the selector: GLM obtains 18.8% on anchored flips and 25.0% on name collisions, while the lifecycle gate is perfect. Implicit commitments are also harder for generic state (42.5% anchored for GLM) than explicit ones (70.0%). These patterns match the predicted distinction between storing evidence and enforcing what that evidence means for denotation.

Human judgments support the core transition, with a fallback boundary. Across 100 blinded items, Fleiss’ $\kappa = .708$, Krippendorff’s $\alpha = .709$, and majority-gold agreement is 86% (91.5% among 94 determinate items); original and rewritten language obtain 84% and 88%. On 50 frozen volunteer rewrites, Generic/CTA/Lifecycle-free/Gated reach 60/90/88/90% for Qwen and 74/98/92/94% for GLM. Direct CTA-minus-Gated intervals are 0.0 [-11.1, +11.1] and +4.0 [-4.3, +14.8] points; CTA-minus-Lifecycle-free intervals also span zero. Against 48 rewrite majorities, CTA reaches 89.6/93.8% versus Gated’s 85.4/85.4%; on 41 majority-actionable items, the values are 92.7/92.7% versus 82.9/82.9%. Lifecycle errors mostly miscompile refresh-first rewrites as PRESERVE; CTA errors concentrate in invalid-target policy. Action-invalid REJECT obtains only 55% majority-gold and 25% unanimity, so we treat Preserve/Reevaluate as the human-supported core and Reject as a normative action policy.

Error and Safety Analysis

Stage analysis localizes the remaining error. On language clusters, GLM compiles mode, ID, and policy correctly on all tasks and has no actor failure. Qwen also compiles all lifecycle fields correctly; its three final errors occur in the dynamic actor. On unseen schemas, mode remains 100%, but bound-ID accuracy falls to 55.0% for explicit and 50.0% for implicit anchored references. The errors concentrate in compound selectors such as “lowest-stock reorderable item” and “oldest failing deployment”: Qwen sometimes chooses the lowest item before filtering *reorderable*, or the oldest deployment before filtering *failing*. The gate then rejects the ineligible ID. This produces conservative errors rather than wrong writes, but it shows that TRI semantics and selector grounding are separate research problems.

Transfer and real write consequences. On the four unseen schemas, lifecycle control improves Qwen from 46.2% to 82.5%; the cluster interval excludes zero. In SQLite replay, Generic Structured Ledger gives Qwen 27/40 final states with 13 wrong-entity writes, while the GLM replay gives 26/40 with 8 wrong-entity writes. Lifecycle-Gated reaches 40/40 for both models with no wrong write, invalid attempt, unnecessary rejection, or collateral modification. The paired exact test for Qwen gives $p = 2.44 \times 10^{-4}$; the GLM paired cluster difference is reported in Table 3. These are actual post-refresh database mutations, not target-string labels. Generic plus a validity-only gate still produces 13 Qwen wrong writes because replacement targets are often action-valid; both Lifecycle variants obtain 40/40. This demonstrates a correctly compiled contract, not general reliability against compiler or tool failures.

Discussion

Commitments benefit from explicit controller state. Generic already carries enough evidence in principle; its failures instead show that an implicit relationship remains behaviorally unstable: is a selector still an instruction to search, or provenance for a committed ID? Pre-refresh compilation answers this once. Typed lifecycle state makes the

Benchmark	Model	Generic	CTA	Lifecycle/Guarded	Δ	Template-cluster 95% CI
Language clusters (primary)	Qwen3.5	64.4	95.0	98.1	+33.8	[+18.1, +50.0]
Language clusters	GLM-5.1	71.9	96.2	100.0	+28.1	[+18.1, +38.1]
Unseen schemas	Qwen3.5	46.2	—	82.5	+36.2	[+25.0, +47.5]
Model-facing SQLite	Qwen3.5	67.5	—	100.0	+32.5	[+15.0, +50.0]
Model-facing SQLite	GLM-5.1	65.0	—	100.0	+35.0	[+17.5, +52.5]
Guarded conditional policies	Qwen3.5	52.5	—	85.0	+32.5	[+15.0, +52.5]

Table 3: Exact target or final-state accuracy (%). CTA is shown where it was run on the identical scalar language inventory; Δ remains Lifecycle/Guarded minus Generic. Confidence intervals resample complete language-template clusters.

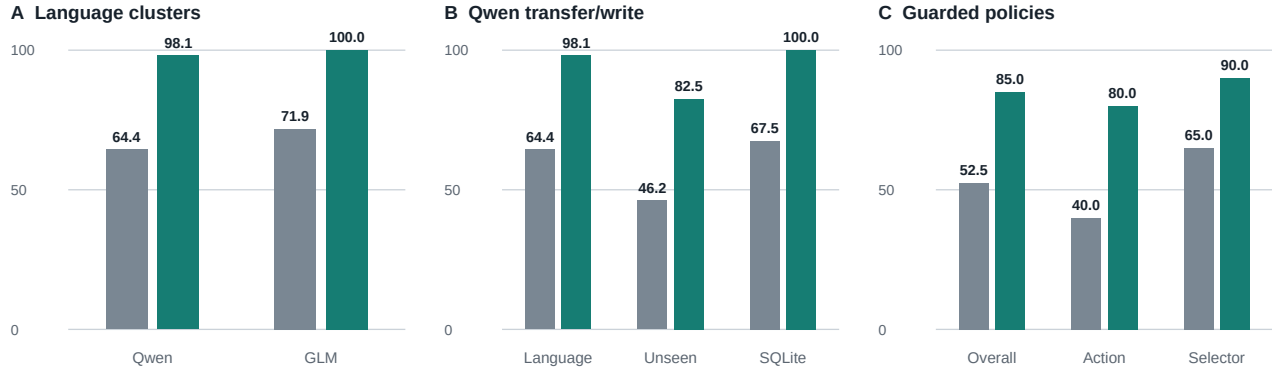


Figure 2: Gray bars are Generic Structured Ledger; green bars are lifecycle-aware controllers. (A) two-model language-cluster replication; (B) Qwen transfer and real SQLite mutation; (C) guarded conditional policies, split by guard semantics.

answer auditable and separates revisable beliefs from discourse commitments without claiming that the tuple is the only sufficient representation.

Gates move, rather than erase, the reliability bottleneck.

A gate cannot repair a wrong mode, ID, or guard; an incorrectly bound yet valid ID can pass. It only prevents a later actor from violating a correctly compiled contract and enforces action validity. On unseen schemas this sometimes converts grounding errors into rejection, shifting rather than eliminating risk and motivating clarification and executable selectors.

Relationship to general tool environments. SQLite prescribes query and refresh; ToolSandbox adds tool choice and compiler errors. In 24 tasks, Gated has no stable advantage and permits one GLM and two Qwen wrong writes, so a gate cannot compensate upstream failures.

Compositional and efficiency boundaries. In Qwen two-refresh/multi-referent stress, Generic reaches 80.0% and scalar Lifecycle 70.0%. Role indexing improves Qwen 35/40 to 39/40 but not GLM consistently. Gating reduces primary requests from 320 to 240 per model.

Limitations

Scope and construct validity. TRI is a controlled diagnostic, not a prevalence estimate. It still simplifies ambiguity, repair, partial observability, and identity migration. Human

validation covers 100 scalar original/rewrite items, not conditional, collection, or nested semantics; low agreement on invalid-target rejection limits fallback claims. Primary experiments use one target and one refresh, while a negative multi-referent stress test does not establish composition.

Causal attribution and external validity. Twenty template clusters and temperature-zero calls do not characterize execution variance or all model families. Component audits are post-primary; compound-selector grounding remains a bottleneck. Fixed query-refresh order, local SQLite, and a mixed ToolSandbox pilot do not establish autonomous AppWorld or real-traffic performance.

Conclusion

World updates need not rebind targets. Across controlled and volunteer-rewritten tasks, minimal pre-refresh CTA has the highest or tied scalar point estimate, without a significant advantage over typed variants. Typed lifecycle state exposes richer policy but adds compilation errors; gates enforce only correct contracts. Composition and external validity remain open. The supported design principle is narrow: decide whether a description has become an identity commitment before incorporating later observations. The results do not establish a universal lifecycle schema or an autonomous-agent safety guarantee.

References

- Babu, R. S.; and Indukuri, S. 2026. Entity Binding Failures in Tool-Augmented Agents. arXiv:2606.30531.
- Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1946–1965.
- Cheng, Y.; Moakhar, A. S.; Fan, C.; Hosseini, P.; Faghieh, K.; Sodagar, Z.; Wang, W.; and Feizi, S. 2026. Your LLM Agents Are Temporally Blind: The Misalignment Between Tool Use Decisions and Human Time Perception. Findings of ACL 2026, arXiv:2510.23853.
- Dasigi, P.; Liu, N. F.; Marasović, A.; Smith, N. A.; and Gardner, M. 2019. Quoref: A Reading Comprehension Dataset with Questions Requiring Coreferential Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 5925–5932.
- Gardner, M.; Artzi, Y.; Basmova, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; et al. 2020. Evaluating Models’ Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1307–1323.
- Grosz, B. J.; Joshi, A. K.; and Weinstein, S. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.
- Ji, H.; Xiong, K.; Han, S.; Xia, P.; Qiu, S.; Zhou, Y.; Liu, J.; Li, J.; Li, B.; Zheng, Z.; Xie, C.; and Yao, H. 2026. ClawArena: Benchmarking AI Agents in Evolving Information Environments. arXiv:2604.04202.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*.
- Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.
- Kamp, H.; and Reyle, U. 1993. *From Discourse to Logic: Introduction to Modeltheoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory*. Kluwer Academic Publishers.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*.
- Koh, J. Y.; Lo, R.; Jang, L.; Duvvur, V.; Lim, M. C.; Huang, P.-Y.; Neubig, G.; Zhou, S.; Salakhutdinov, R.; and Fried, D. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- Lee, K.; He, L.; Lewis, M.; and Zettlemoyer, L. 2017. End-to-end Neural Coreference Resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 188–197.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.
- Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.
- Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.
- Mialon, G.; Fourrier, C.; Swift, C.; Wolf, T.; LeCun, Y.; and Scialom, T. 2024. GAIA: A Benchmark for General AI Assistants. In *International Conference on Learning Representations*.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Park, J. S.; O’Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
- Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.
- Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–4912.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.
- Sohail, S.; and Haider, G. 2026. Bounded Autonomy for Enterprise AI: Typed Action Contracts and Consumer-Side Execution. arXiv:2604.14723.
- Sumers, T. R.; Yao, S.; Narasimhan, K.; and Griffiths, T. L. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*.
- Tang, Z.; Zhao, Q.; Franco, G.; Wijaya, D.; Mueller, A.; Schuster, S.; and Kim, N. 2026. Do Language Models Track Entities Across State Changes? In *International Conference on Machine Learning*. ArXiv:2605.30233.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Li, V. D.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv:2407.18901.
- Uddin, M. N.; Saeidi, A.; Blanco, E.; and Baral, C. 2026. Ledger-Agent: Structured State for Policy-Adherent Tool-Calling Agents. arXiv:2606.20529.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Wang, X.; Wang, Z.; Liu, J.; Chen, Y.; Yuan, L.; Peng, H.; and Ji, H. 2024. MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. In *International Conference on Learning Representations*.

Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

Xu, J.; Li, Q.; Wu, J.; Lan, Y.; Li, S.; Zhou, H.; Jiang, B.; Wang, L.; Wang, J.; Luu, A. T.; Xiong, C.; Park, H. W.; Hooi, B.; and Hu, Z. 2026. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. arXiv:2606.13681.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zhang, Z.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Dai, Q.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2024. A Survey on the Memory Mechanism of Large Language Model Based Agents. arXiv:2404.13501.

Zhong, W.; Guo, L.; Gao, Q.; Ye, H.; and Wang, Y. 2024. Memory-Bank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.